Alex Castillo González

Applied AI Engineer · Python / LLM

Remote — UTC−4 · Spanish (native), English (professional) alex.castillog33@gmail.com · github.com/pcbeingused333 · linkedin.com/in/alex-castillo-gonzalez · portfolio-alexgonzalez33.vercel.app

SUMMARY

Applied AI engineer working in Python on retrieval and agent systems, and on the layer that decides whether they survive real users: evaluation, failure handling, and knowing which numbers actually moved. My main retrieval project answers over the text of the **GDPR** and is built around a constraint that regulated domains impose and generic RAG ignores — every statement has to name the provision it came from, and the system has to decline when the source does not cover the question. Both projects ship with the harness that measures them — retrieval, citation accuracy and abstention for the RAG system, tool trajectories and answer grounding for the MCP agent — and in each case the harness found defects the tests did not. Fullstack background across Python, TypeScript and Ruby, with production experience shipping and operating what I build. Two merged upstream contributions to `pyfenn/fenn`, a Python framework for ML workflows and LLM agents, and three open fixes to the retrieval evaluation and MMR code in `llama-index-core`.

SKILLS

AI / LLM — Python · LLM APIs (Groq, OpenAI-compatible) · LangChain · LangGraph · ReAct agents and tool use · **Model Context Protocol (MCP)**: building servers and clients · **agent trajectory evaluation** (tool selection, ordering, argument accuracy, answer grounding) · RAG: chunking strategy, retrieval tuning, **structure-aware citation at the provision level** · **LLM evaluation**: known-answer datasets, retrieval metrics (hit@1, recall@k, MRR), **abstention and unsupported-claim rate**, LLM-as-judge for faithfulness / relevancy / correctness / context precision · cross-lingual retrieval · resilience to unreliable model output (retry with rate-limit backoff, graceful degradation)

Regulatory / legal text — parsing legislative sources into citable provisions (EUR-Lex / Official Journal markup) · citation integrity as a design constraint · evaluating refusal on out-of-corpus questions · GDPR structure (data subject rights, controller and processor obligations, breach notification, administrative fines)

Data & retrieval — embeddings (`sentence-transformers`, BGE, MiniLM) · FAISS · PostgreSQL + `pgvector` · PDF ingestion pipelines · structured corpus construction

Backend — Python · pytest · Ruby on Rails · PostgreSQL · REST APIs · JSON/API integrations (Jira API)

Web — TypeScript · Next.js · React · Tailwind · JavaScript · HTML/CSS/SCSS

Infra & tooling — **AWS** (Lambda container images, DynamoDB, ECR, IAM least-privilege, CloudWatch, Budgets) · **Terraform** · **CI/CD with GitHub Actions, OIDC federation** · Docker · Docker Compose · Vercel · Streamlit Community Cloud · Git (GitHub, GitLab) · AI-assisted development (Claude Code, Cursor)

EXPERIENCE

Churrería Calderón — Family business · Toronto, Canada

Oct 2025 – Jul 2026 *(business closed July 2026)*

- **Developer:** built and deployed the business website, plus an embeddable AI chat widget that answered customer questions on menu, hours, location and FAQs, grounded only in the business's own information so it would not invent details. Built the widget to be reusable across clients from a single configuration file. Next.js, React, TypeScript, Tailwind, Groq, Vercel.
- Shipped both during the setup period before the December 2025 opening, so the site and the assistant were live on day one rather than added later.
- Worked in day-to-day operations of the business throughout, which is where the judgement about which problems are worth automating — and which are not — came from.

Family churrería business — Owner-operator · Catalonia, Spain

2023 – 2026 *(and in the same business before that)*

- Ran the business single-handed: production, service, customers, purchasing, cash and compliance. No staff to delegate to and no manager to escalate to — if it did not work, it was mine to fix that morning.
- Took it from a mobile trailer to fixed premises, which meant rebuilding the operation around a different site, different hours and a different customer base.
- Also worked my father's stand on the fairground circuit.
- Years of reading what customers actually ask for, in person, hundreds of times a day. That is the same instinct a support-facing product feature needs, and it is why the AI widget I later built was scoped to the four questions people really ask.

Le Wagon — Part-time Programming Teacher · Remote (France)

Oct 2022

- Taught on the new part-time flex cohort of the fullstack bootcamp: Ruby, object-oriented programming, SQL and PostgreSQL, HTML/CSS/JavaScript, and building on Ruby on Rails.
- Supported students through exercises and debugging during live sessions.

TECNOBIT (Grupo Oesía) — Fullstack Developer · Valdepeñas, Spain

Aug 2022 – Nov 2022 · On-site

- Shipped features into a long-running internal application maintained by a team of four engineers and a systems engineer.
- **Backend:** wrote Ruby routines that pulled data from JSON files and the **Jira API**, transformed it and persisted it to PostgreSQL; implemented multi-stage validation processes that gated document generation and downloads.
- **Frontend:** built form-driven pages and PostgreSQL-backed views with role-dependent data (users and managers saw different data), and surfaced the state of backend subprocesses so users could follow a document download to completion.
- Worked across a codebase distributed over both GitHub and GitLab.

SELECTED PROJECTS

Business Ops Agent — MCP server + agent, with trajectory evaluation

Live demo · Code

A **Model Context Protocol server** exposing a business's operations (catalog, booking capacity, stock, quoting, orders) as tools any MCP client can call — Claude Desktop, Cursor, or the LangGraph agent bundled with it. The agent carries no business rules; tools are discovered at runtime, so adding one requires no agent change.

- Built an evaluation harness that scores **tool trajectories**, not just answers: which tools were called, in what order, with which arguments, and whether every figure in the reply traces back to a tool result. The grounding check needs no judge model and is therefore deterministic and free — it holds even when a fabricated number happens to be correct.
- The harness caught the agent answering "I don't have that information" with zero tool calls, and caught it **intermittently**, which single-run testing misses; scenarios can be repeated to measure flaky behaviour. Scored 11/12, mean 0.98.
- Chose not to use `langchain-mcp-adapters`: it pins `mcp<2` and would have forced a working server back to an older SDK. Wrote a 60-line bridge instead, passing the server's JSON Schema straight through so no tool signature is duplicated.
- **Deployed it to AWS** as a remote MCP server — Lambda container behind a Function URL, DynamoDB single-table store, least-privilege IAM, all in Terraform. Storage sits behind an interface, so the same server runs on SQLite locally and DynamoDB in production with no change above the backend.
- CI/CD on every push to main via GitHub Actions, authenticating with **OIDC** rather than a stored access key, and scoped to one branch so a fork's pull request cannot assume the deploy role. The role deliberately cannot apply infrastructure: it can ship an image and repoint the function, nothing more.

- Python, MCP 2.0, LangGraph, Groq, AWS (Lambda, DynamoDB, ECR, IAM), Terraform, Docker, GitHub Actions, pytest (161 tests).

Ask the GDPR — retrieval over regulation, with citations that can be checked

Live demo · Code

LangGraph agent over the full text of the GDPR. Every answer names the provision behind it — `Art. 33(1)`, not a page number — and the system is built to decline when the source does not cover the question. Two modes behind one flag: an in-memory FAISS demo on a free 1 GB container, and a pgvector-backed production path.

- **Made the citation structural rather than incidental.** A regulation is cited by article and paragraph; the page a provision lands on is an artefact of typesetting, and a reader sent to "page 14" can confirm nothing. So the corpus is not a PDF: a builder parses the Official Journal text from EUR-Lex (not a mirror — in a system whose claim is checkable citations, the text has to come from the authority the citation names) into **414 provisions** carrying article, paragraph, title and chapter as metadata. The chunk size was then chosen so that **97% of provisions survive as exactly one chunk**, because a chunk straddling Art. 33(1) and 33(2) gets attributed to one of them and cites the wrong paragraph.
- **Built an eval for the answers that should never be given.** In legal text a retrieval miss announces itself; an invention is fluent, confident and indistinguishable from a correct answer, and a citation the model reasoned its way to rather than read makes it more convincing. So the harness scores refusal against questions the Regulation does not answer but every model has read about — adequacy decisions by country, the text of the standard contractual clauses, Schrems II, a CCPA penalty — alongside a deterministic check for citations that appear in the answer but never in the retrieved passages. Every question scored so far declined correctly and specifically, naming the provision that creates the mechanism and stating that the detail asked for is not in the text; none produced an unsupported answer or a citation that was never retrieved.
- Changed the ground truth from matching text to **matching the cited provision**, which is stricter (retrieving the right words is not retrieving the right authority) and removes the chunk-boundary false misses the substring approach suffered.
- **Re-ran every measurement when the corpus changed, and one result reversed.** Embedding the article heading measurably *hurt* retrieval under the old embedding model and measurably *helped* under the new one; carrying the first conclusion forward would have shipped the worse setting on the strength of real evidence. The model swap itself was worth 8/20 → 13/20 at rank 1 for 42 MB of RAM, against a hard 1 GB ceiling measured with the index loaded, not just the model.
- Three parse bugs caught by sanity checks that now block the build: the closing article silently absorbing the signatures and all 21 footnotes, nested sub-points truncated by a non-greedy regex that could not handle nested markup, and the 26 definitions of Art. 4 collapsing into one uncitable record.
- Fixed cross-lingual retrieval (Spanish 0/4 at rank 1 against an English index) by translating the retrieval query rather than paying ~350 MB for a multilingual embedding model that did not fit the 1 GB budget — a measured trade-off.
- CI on every push runs the suite headlessly, including a boot test of the app itself — the host redeploys straight from `main`, so the suite is the only gate before the public demo. One test forces the embedding loader to raise and asserts the first render still succeeds, proving nothing heavy sits on the render path.
- Python, LangChain, LangGraph, Groq, FAISS, pgvector, Streamlit, Docker, GitHub Actions, pytest (72 tests).

AI Website Chat Widget — embeddable business assistant

Live demo · Code

Drop-in chat widget for small-business sites, grounded strictly in the business's own content. Reusable for any client from a single config file. Next.js, React, TypeScript, Tailwind, Groq, Vercel.

Semantic Recommender — embedding-based recommendations

Code

Recommendation engine built on vector embeddings with a feedback loop that refines results over time, as a reusable backend for platforms that have outgrown rule-based filters. Python, embeddings, pgvector, PostgreSQL.

OPEN SOURCE

Merged

- `pyfenn/fenn` #277 — added `.docx` support to the RAG document loader, so the framework ingests Word documents alongside PDFs and text.
- `pyfenn/fenn` #286 — corrected the RAG optional-dependency install instructions, which referenced a package name that does not exist.

Open

- `run-llama/llama_index` — three fixes in `llama-index-core`, found by reading the retrieval and evaluation code rather than from an issue. #22683: the retrieval metrics scored outside their own range when a ranking repeated a node id, which is what fusion retrievers produce — hit rate and average precision returned 2.0, NDCG 1.63, so a mean over an eval set stopped being comparable between runs. #22684: MMR discounted each candidate only against the result selected immediately before it, so a near-duplicate re-entered the ranking as soon as an unrelated result was picked in between. #22685: the multi-modal evaluator scored image nodes as text results, because `ImageNode` subclasses `TextNode` and the two type checks were independent. Each ships with a test that fails without the fix.
- `rubocop/rubocop-rspec` #2209 — fixed a crash in `RSpec/LeadingSubject` on Ruby 3.4's implicit `it` block parameter: the cop only walked `:block` AST ancestors and hit `nil` on the new `itblock/numblock` nodes. Widened the lookup to `:any_block`, with a regression spec.
- `rubocop/rubocop-rspec` #2214 — fixed `RSpec/LeadingSubject` autocorrecting a subject to a position above another subject.
- `rubocop/rubocop-performance` #529 — fixed `Performance/ConstantRegexp` emitting invalid code when autocorrecting a regexp used as a pattern in `case/in` pattern matching.
- `Rails-Designer/courrier` — four PRs: MailerSend, Mailtrap and SMTP.com provider integrations, and a `NameError` fix affecting Mailgun and Mailjet on Ruby 3.4.

EDUCATION

Le Wagon — Fullstack Web Development bootcamp · 2022 Ruby, Ruby on Rails, JavaScript, SQL/PostgreSQL, HTML/CSS.

Self-directed, 2022 – 2025 — LaunchSchool coursework, coding challenges and independent study alongside running the business, before moving back into engineering full time.